PRACTICAL 4

Aim :

Build a Node/Express backend for a Task Management system. Implement REST endpoints for creating, reading, updating, and deleting tasks (in-memory array, no database yet). Apply a request logging middleware that logs method, URL, and timestamp for every incoming request. Apply a global error handling middleware as the last middleware in the pipeline. Use correct HTTP status codes (200, 201, 404, 500) for each response.

Code :

```
const express = require("express");


const app = express();
const PORT = 5000;


// ----------------------------------------------------
// In-memory task storage
// ----------------------------------------------------


let tasks = [
  {
    id: 1,
    title: "Complete AWDF Practical 4",
    completed: false,
  },
  {
    id: 2,
    title: "Test REST API using Postman",
    completed: false,
  },
];


// ----------------------------------------------------
// Built-in middleware
// ----------------------------------------------------
```

```
app.use(express.json());

// ----------------------------------------------------
// Logging middleware
// ----------------------------------------------------

app.use((req, res, next) => {
  console.log(
    `${req.method} ${req.url} - ${new Date().toISOString()}`
  );

  next();
});

// ----------------------------------------------------
// Content-Type validation middleware
// Applies to POST and PUT
// ----------------------------------------------------

app.use((req, res, next) => {
  if (req.method === "POST" || req.method === "PUT") {
    const contentType = req.headers["content-type"];

    if (
      !contentType ||
      !contentType.includes("application/json")
    ) {
      return res.status(400).json({
        error: "Content-Type must be application/json",
      });
    }
  }
```

```
  next();
});

// ----------------------------------------------------
// Task ID validation middleware
// ----------------------------------------------------

const validateTaskId = (req, res, next) => {
  const id = Number(req.params.id);

  if (!Number.isInteger(id) || id <= 0) {
    return res.status(400).json({
      error: "Task ID must be a positive integer",
    });
  }

  next();
};

// ----------------------------------------------------
// GET /tasks
// Get all tasks
// ----------------------------------------------------

app.get("/tasks", (req, res) => {
  res.status(200).json(tasks);
});

// ----------------------------------------------------
// POST /tasks
// Create a new task
// ----------------------------------------------------

app.post("/tasks", (req, res, next) => {
```

```
  try {
    const { title, completed = false } = req.body;

    if (!title || typeof title !== "string") {
      return res.status(400).json({
        error: "Task title is required",
      });
    }

    const newTask = {
      id: tasks.length > 0
        ? tasks[tasks.length - 1].id + 1
        : 1,
      title,
      completed: Boolean(completed),
    };

    tasks.push(newTask);

    res.status(201).json(newTask);
  } catch (error) {
    next(error);
  }
});

// ----------------------------------------------------
// PUT /tasks/:id
// Update a task
// ----------------------------------------------------

app.put("/tasks/:id", validateTaskId, (req, res, next) => {
  try {
    const id = Number(req.params.id);
```

```
const taskIndex = tasks.findIndex(
  (task) => task.id === id
);

if (taskIndex === -1) {
  return res.status(404).json({
    error: "Task not found",
  });
}

const { title, completed } = req.body;

if (title !== undefined && typeof title !== "string") {
  return res.status(400).json({
    error: "Title must be a string",
  });
}

if (
  completed !== undefined &&
  typeof completed !== "boolean"
) {
  return res.status(400).json({
    error: "Completed must be a boolean",
  });
}

tasks[taskIndex] = {
  ...tasks[taskIndex],
  ...(title !== undefined && { title }),
  ...(completed !== undefined && { completed }),
};

res.status(200).json(tasks[taskIndex]);
```

```
  } catch (error) {
    next(error);
  }
});

// ----------------------------------------------------
// DELETE /tasks/:id
// Delete a task
// ----------------------------------------------------

app.delete("/tasks/:id", validateTaskId, (req, res, next) => {
  try {
    const id = Number(req.params.id);

    const taskIndex = tasks.findIndex(
      (task) => task.id === id
    );

    if (taskIndex === -1) {
      return res.status(404).json({
        error: "Task not found",
      });
    }

    const deletedTask = tasks.splice(taskIndex, 1)[0];

    res.status(200).json({
      message: "Task deleted successfully",
      task: deletedTask,
    });
  } catch (error) {
    next(error);
  }
});
```

```
// ------------------------------------------------
// 404 handler for undefined routes
// ------------------------------------------------

app.use((req, res) => {
  res.status(404).json({
    error: "Route not found",
    path: req.originalUrl,
  });
});

// ------------------------------------------------
// Global error handling middleware
// MUST BE LAST
// ------------------------------------------------

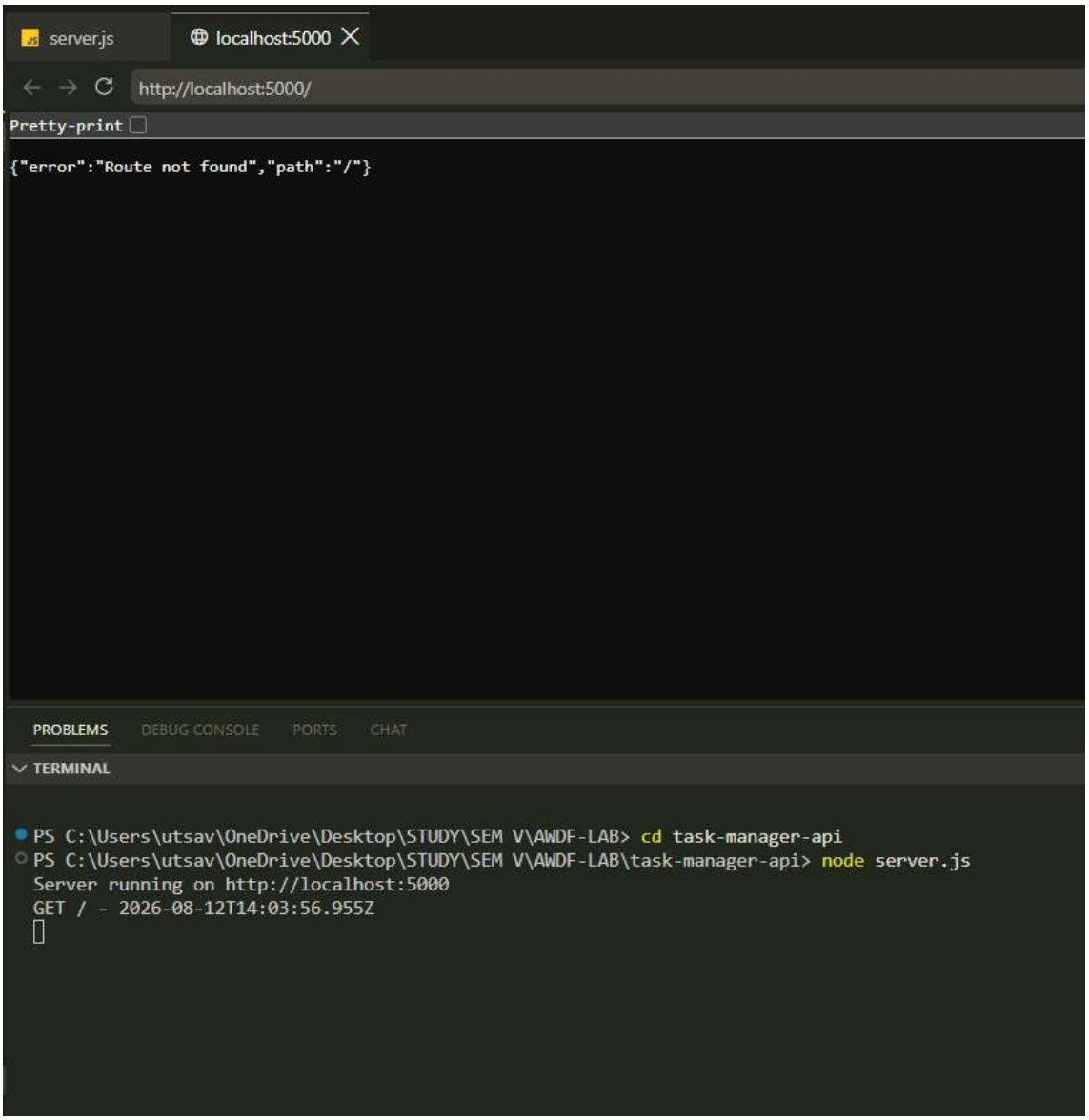
app.use((err, req, res, next) => {
  console.error(err.stack);

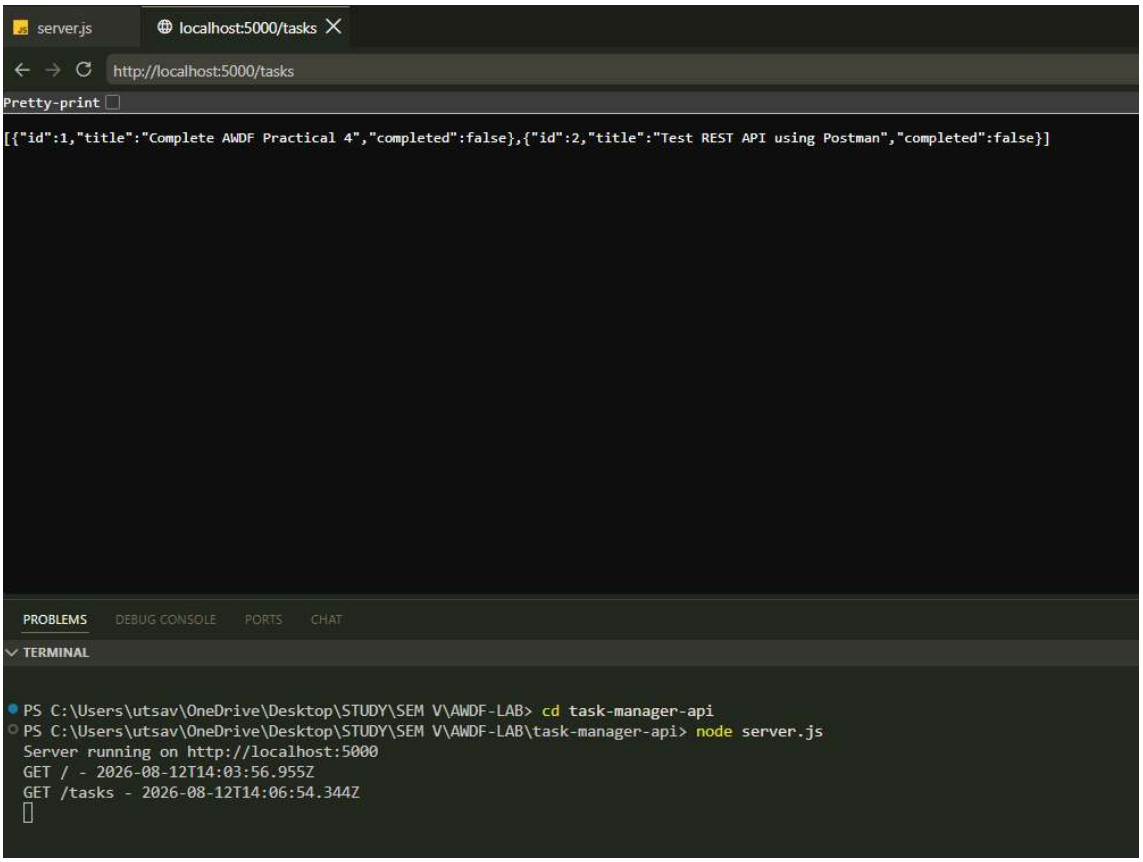
  res.status(500).json({
    error: "Something went wrong",
  });
});

// ------------------------------------------------
// Start server
// ------------------------------------------------

app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

Output :


server.js
localhost:5000
http://localhost:5000/
Pretty-print
{"error":"Route not found","path":"/"}
PROBLEMS
DEBUG CONSOLE
PORTS
CHAT
TERMINAL
PS C:\Users\utsav\OneDrive\Desktop\STUDY\SEM V\AWDF-LAB> cd task-manager-api
PS C:\Users\utsav\OneDrive\Desktop\STUDY\SEM V\AWDF-LAB\task-manager-api> node server.js
Server running on http://localhost:5000
GET / - 2026-08-12T14:03:56.955Z


server.js
localhost:5000/tasks
http://localhost:5000/tasks
Pretty-print
[{"id":1,"title":"Complete AWDF Practical 4","completed":false},{"id":2,"title":"Test REST API using Postman","completed":false}]
PROBLEMS
DEBUG CONSOLE
PORTS
CHAT
TERMINAL
PS C:\Users\utsav\OneDrive\Desktop\STUDY\SEM V\AWDF-LAB> cd task-manager-api
PS C:\Users\utsav\OneDrive\Desktop\STUDY\SEM V\AWDF-LAB\task-manager-api> node server.js
Server running on http://localhost:5000
GET / - 2026-08-12T14:03:56.955Z
GET /tasks - 2026-08-12T14:06:54.344Z